

Cheat sheet — everything for the exam, one file

Built from: the 2026 endterm result sheet, your *Final Guide*, the *if-in-test*, *black-box* and *BIE setup* sheets, the component-diagram picture, and the two code templates. Copy-paste templates for every kind of task (implement, JUnit, parameterized, EasyMock x2, REST impl, REST test, black-box, Gherkin): see [templates/README.md](#). The original PDFs are in [pdfs/](#). Your long lecture-based reference, searchable with Ctrl+F tags (`#easymock`, `#rest`, `#bva`, `#quizbank` ...), is [ISE_Master_Cheatsheet.md](#).

0. Before and during the exam

Setup (BIE machine, ~5 min wait after login): login (*Anderer Benutzer* → *TUM_ID* → *password*) → IntelliJ IDEA Community 2023.1.7, **don't import settings**, accept terms → *Get from VCS* (install Git if missing, restart IDE) → clone any exercise from Artemis (practice mode) → **Project Structure** → **download JDK** → **Amazon Corretto 17**, **language level 17** → *Settings* → *Gradle* → *Gradle JVM* = *corretto-17* → make a change, **commit**, **push**, **enter your email** → open Safe Exam Browser.

The rules that decide pass/fail (from three real result sheets):

1. `git add .` && `git commit -m "start"` && `git push` **before anything else**, then after every endpoint / every test. Artemis grades the **pushed** commit. Unpushed = 0. (Two earlier exams lost 20 + 22 points to "no commit was made".)
 2. **Quiz first (15 min), REST second.** REST is the biggest block (≈ 2.8 pts per endpoint-test).
 3. **Never leave** `fail("Test is not implemented yet")`. Half-finished = real points; unstated = 0.
 4. **Never rename a test method or a class.** The grader matches names.
 5. Copy names/words **character-for-character** from the task (`SHORT_HAUL`, `«machine»`, `ticketId`).
 6. Ask of every test: *if I deleted the body of the method I test, would this test fail?* If not it scores 0.
-

1. JUnit 5

```
class XTest {
    private X x;
    @BeforeEach void setUp() { x = new X(50.0, Tier.A, 2); }           // fresh object per test

    @Test void testSomething() {
        // ARRANGE (numbers by hand from the task)  ACT (one call, result CAPTURED)  ASSERT
        (everything)
        AllocationResult r = x.allocate(Tier.A, 5.0);                 // <- assign it!
        assertTrue(r.isSuccess());
        assertEquals(3.0, r.getCost(), 0.0001);                       // expected FIRST,
        delta on doubles
        assertEquals(47.0, x.getRam(), 0.0001);                       // and the STATE it
        changed
    }
    @Test void testThrows() { assertThrows(IllegalArgumentException.class, () ->
        x.method(null)); }
}
```

Assertion	Use for
<code>assertEquals(exp, act)</code>	int, String, enum, counts. Objects need equals() — without it you compare references and never pass; assert on getters instead
<code>assertEquals(exp, act, delta)</code>	double / float — always
<code>assertTrue / assertFalse</code>	boolean results
<code>assertNull / assertNotNull</code> , <code>assertSame(a, b)</code>	null checks / same reference
<code>assertThrows(Ex.class, () -> call())</code>	error branch — the call goes inside the lambda; never try/catch it
<code>assertAll(...)</code>	run several, report every failure
<code>@BeforeEach/@AfterEach</code> · <code>@BeforeAll/@AfterAll (static)</code> · <code>@Disabled</code>	lifecycle / skip

Seven tests to write for any "implement then test" class (matches `TestTemplate`): normal case · other branch (the `else`) · failure case (**state unchanged**) · cap/limit (`min/max`, check return **and** stored state) · **exact boundary** (`ram == cost` must succeed when the check is `<`) · invalid input (0 / negative → nothing changes) · exception (only if the task says one is thrown).

Traps: the result of the method under test **must be stored** (`AllocationResult r = server.allocateTask(...)`) — in 2026 one missing line killed 4 tests · integer division (`160/15 = 10`; write `80.0`) · `%` is not a cap, `Math.min` is · compare a speed to a speed, not to a cost · remember the *starting* value (`10 + 13 = 23`, not 13) · failed path must not mutate (compute → check → mutate) · `assertTrue(result == 32.0)` **is** a correct check when the value is exactly representable.

"if" in a test class — two different things

	What it is
<code>assumeTrue(cond) / assumeFalse(cond)</code>	Skips the test when the assumption fails. Skipped ≠ passed ≠ failed (a third result category).
<code>plain if (...) { assertA } else { assertB }</code>	Normal Java branching inside a <code>@Test</code> . Valid but bad practice (one test, two scenarios). Fix: split into two @Test methods , one per branch.

For CFG / complexity questions treat it exactly like production code: **V(G) = decision points + 1**; every `if`, `else if`, `for`, `while`, `case`, `&&`, `||` is +1. One `if/else` needs ≥ 2 test cases for branch coverage.

2. EasyMock (only if mocking appears — it was absent in 2026)

Steps, always in this order: **create** → **record** → **replay** → **verify**. Template: `templates/MockTemplate.java`.

```

@ExtendWith(EasyMockExtension.class) // annotation style
class Test {
    @TestSubject Sut sut = new Sut(); // must be INSTANTIATED inline
    @Mock Collaborator mock; // leave unassigned; the extension
injects it
    @Test void t() {
        expect(mock.f("a")).andReturn(2.0); // record, value
        mock.g("a"); expectLastCall(); // record, void
        expect(mock.h()).andThrow(new IllegalStateException());
        replay(mock); // forget this -> "missing behavior"
definition"
        sut.run(); // CALL THE SUT
        verify(mock); // forget this -> missing calls
undetected
    }
}

```

Need	Write
called twice / 1-3 / any	<code>.times(2) · .times(1,3) · .anyTimes()</code> (includes 0) · <code>.atLeastOnce()</code> · <code>.once()</code>
different value per call	<code>.andReturn(2.0).andReturn(3.0)</code>
any argument	<code>anyString()</code> , <code>anyInt()</code> — if one arg is a matcher, all must be : <code>foo(anyInt(), eq(5))</code>
must never be called	do not record it, keep a default mock, <code>verify</code>
order matters	<code>createStrictMock</code>
call happened <i>and</i> effect	<code>verify(mock)</code> plus a state assertion on the real object

Doubles: **Dummy** (fills a parameter) · **Stub** (canned answers) · **Spy** (records calls) · **Mock** (expectations, verified) · **Fake** (simplified real implementation).

3. REST with Spring (25 pts)

```

@RestController @RequestMapping("/tickets")
public class TicketResource {
    private final TicketService ticketService;
    public TicketResource(TicketService s) { this.ticketService = s; } // constructor
injection

    @PostMapping                      ResponseEntity<Ticket> create(@RequestBody Ticket t) //
400 if t.getId() != null
    @GetMapping("/{ticketId}") ResponseEntity<Ticket> get(@PathVariable Long ticketId) //
404 if absent
    @GetMapping                      ResponseEntity<List<Ticket>> all()
    @PutMapping("/{ticketId}") ResponseEntity<Ticket> update(@PathVariable Long ticketId,
                                                                @RequestBody Ticket t) //
400 mismatch, 404 unknown
    @DeleteMapping("/{ticketId}") ResponseEntity<Void> delete(@PathVariable Long ticketId) //
204
}

```

Decision	Rule
Does <code>@RequestMapping</code> on the class already carry a path?	If it sets only consumes/produces, every method spells out the full path — otherwise every test fails at once
Path variable or query param?	Identity → path (<code>/tickets/{id}</code>), filter/sort → query (<code>?onlyOpen=true</code> , <code>@RequestParam(required=false, defaultValue="false")</code>)

Decision	Rule
Verb	create → POST · read → GET · replace → PUT · remove → DELETE. The verb is never in the URL
Reject how?	throw new <code>ResponseStatusException(HttpStatus.BAD_REQUEST)</code> or return <code>ResponseEntity.badRequest().build()</code> — match what the skeleton imports
Status codes	Follow the task's table , not REST habit (POST created = 200 in the 2026 exam). <code>ok()</code> 200 · <code>noContent()</code> 204 · <code>notFound()</code> 404 · <code>badRequest()</code> 400
Layering	Resource = HTTP + validation · Service = rules · Repository = data. Never skip a layer

The four bugs that cost marks in 2026: `@PathVariable Long id` vs `{ticketId}` → **500** (names must match, or `@PathVariable("ticketId")`) · `getId() == ticketId` on boxed `Long` (use `.equals`) · update returning `null` on success · create not rejecting a body with an id.

Service over a list: assign id then `nextId++` · `getAll` returns `new ArrayList<>(tickets)` · find with `.equals` · update copies **every** field into the **stored** object and returns it, `null` if not found · delete with `removeIf`, never throws.

Client half (rarely asked):

```
webClient.post().uri("/x").bodyValue(o).retrieve().bodyToMono(T.class).block() ·
bodyToFlux(T.class).collectList().block() · nothing is sent without .block()/ .subscribe().
```

4. Black-box testing (15 pts)

Three parts, always: equivalence classes → one representative per class → three-point boundaries.

Rule	Detail
Split per bracket, per parameter combination	If limits depend on the device type, you need classes for below / inside / above for each type
"Valid" = accepted and evaluated	A valid input may produce <code>rejected/unsupported</code> -style output; that is not "Invalid"
One catch-all Invalid class per parameter	"any other value or empty input"
Give every class an id, and write a coverage note	TC1–TC3 cover ... is its own scoring criterion
Minimal set	one test per class, only one invalid parameter at a time · never leave a <fill> or a cell blank (a blank is 0) · expected results are exactly the allowed words (<code>performance</code> / <code>quality</code> / <code>unsupported</code>), never a comment
Three-point boundary	three single numbers, not ranges , around the limit named in the heading: L–1, L, L+1 ; L is the first valid value (2047 / 2048 / 2049, not <code>2048 <= vramMB <= 16384</code>). If 1 is the smallest legal value test 0 / 1 / 2
The marks are on the L row	it is the only one that fails when <code>>=</code> is typed as <code>></code>

Quick table version (works on any "input → output" task): bad inputs one row each (negative, too big, wrong type, not a number, empty) · good inputs (normal, any special value) · edges **limit–1, limit, limit+1**. Always include the **Expected Output** column. Testing 5 *and* 7 is one row, not two.

Worked, verified answers: `ise.blackbox.exam_graphics` (2026), `ise.blackbox.exam_gamelauncher` (mock), `ise.blackbox.practice_streaming`.

5. Modeling

Deployment diagram — what physically runs where

Element	Drawn as	Rule
Node	3-D box	A flat rectangle at top level is marked wrong
«device»	node stereotype	physical hardware (phone, sensor, badge, terminal)
«machine» / «executionEnvironment»	node stereotype	use the word the statement uses («node» ≠ «machine» cost a point)
Artifact	box «artifact» inside the node	.jar, .sql, "Cart UI". Nesting = "deployed on"; no extra arrow
Component	box «component» inside a node	software unit
Communication path	plain line node↔node, no arrowhead , label = protocol (TLS, NFC, gRPC, MQTT, HTTPS, Wi-Fi)	one line per "communicates with" sentence
Interface	lollipop ○— (provided) + socket —○ (required), component↔component , labelled with the service name	never "Interface"; drawing TLS as a lollipop is a real mistake

Draw everything the statement lists, and only that — in 2026 only a few boxes were drawn and 16 of 20 points were lost. «artifact» for "runs ... artifact", «component» for "deploys ... component", and a communication path is a plain line, **never a lollipop**.

Build order: nodes → nest artifacts/components → paths with protocol labels → read the Hint → interfaces named from the text → spell-check stereotypes. Add **nothing** the statement does not mention.

The component-diagram picture you sent (smart home) follows exactly this: Smartphone → *HTTPS* → Cloud Server (API Gateway, User Authentication Service, Device Control Service); Database Server (User Accounts, Device Registry, Device State History) provides interfaces that the Cloud Server's components require; Cloud Server → *MQTT* → Home Hub (Hub Agent) → *Wi-Fi* → Smart Light («device»); Smartphone (Billing Client) ↔ Bank Server over *TLS* via an interface. The three green ticks mark what was graded correct; every interface there should be named after its service instead of "Interface".

Other diagrams (one line each)

- **Component ≈ Deployment** notation (lollipop/socket) — learn together; Component was the diagram set most often in the 2026 course.
 - **Class:** hollow triangle + solid line = generalisation (to parent) · hollow triangle + dashed = realisation (to interface) · dashed open arrow = dependency · hollow diamond = aggregation (part survives) · filled diamond = composition (part dies) · - + # ~ = private/public/protected/package.
 - **Use case:** «include» always runs (arrow → included) · «extend» sometimes (arrow → base).
 - **Activity:** fork → join (join **waits**) · decision/merge (merge doesn't wait) · guards in [brackets].
 - **Communication diagram:** every message **numbered** (1: , 2.1:); objects written name:Type underlined.
 - **Patterns:** Observer (subject attach/detach/notify, observer update) · Strategy (swap algorithm) · Adapter (make an incompatible class fit) · Template Method (skeleton in superclass).
- A model that is not the right pattern scores 0**, not partial.

6. Quiz theory (20 pts, 15 min)

Topic	Remember
Coverage	Statement (every line) < Branch (each decision true <i>and</i> false) < Term/condition (each atomic condition both ways) < Path . 100 % statement ≠ 100 % branch.
Coverage arithmetic	count executable lines (not signatures / lone <code>}</code>), run each test, union the lines, covered ÷ total. <code>if</code> line executes whether true or false; its body only if taken.
Cyclomatic complexity	decisions + 1 = min. tests for branch coverage. Correlates with size , weakly with understandability. Cognitive complexity rises with broken linear flow and nesting .
Fault chain	Mistake (human) → Fault/Defect (in code) → Error (wrong state) → Failure (observable)
Verification / Validation	built the product right (spec) / built the right product (need)
Test levels	unit (one method in isolation) · integration (components together) · system (whole, production-like, functional and non-functional) · acceptance (user needs)
Test types	performance (speed) · load (expected peak) · stress (beyond, find the break)
Includes	Common = same setup, different features · Parameterized = same logic, different data
Git	working dir → staging → repository · untracked/modified → <code>add</code> → staged → <code>commit</code> → committed · <code>fetch</code> (download) vs <code>pull</code> (= <code>fetch</code> + <code>merge</code>) · <code>checkout -b</code> · <code>push -u origin x</code> · conflict: edit, delete markers, add, <code>commit</code>
Gradle / CI	<code>gradlew build</code> = assemble + check (compile + test) · pipeline Build → Static analysis → Unit → Integration → System → Acceptance (fail fast)
Delivery vs Deployment	Continuous Delivery = manual gate to production · Continuous Deployment = automatic
Release strategies	Dark launching (runs on real traffic, users see nothing) · Canary (small share of users) · Blue-green (switch whole environments) · Feature toggle (flag)
Static analysis (ASA)	inspects code without running it — vulnerabilities, null pointers, before build. Unit tests are dynamic
Reviews	log defects, defer solutions ; no justification-before-logging, no interruptions
Requirements	functional = what · non-functional = how well (usability, performance, reliability, security, safety, accessibility). INVEST : Independent, Negotiable, Valuable, Estimable, Small, Testable ("helpful" is not testable). Epic > Story > Task
Design principles	Open-Closed (open for extension, closed for modification) · Dependency Inversion · information hiding · low coupling / high cohesion
Process models	Incremental = several usable parts, each adds functionality · Waterfall = largely sequential phases · Kanban = board, cards left to right, WIP limit per stage (no fixed sprints) · Scrum : Sprints + Sprint Review; the Scrum Master facilitates and removes impediments, the Product Owner owns the backlog

Topic	Remember
Software characteristics	software has no natural locality , so SE must create distance and isolation (memory does <i>not</i> do it) · reuse is lucrative only when little customisation is needed — it is not always cheap
Models	Mapping (represents an original) · Abstraction/Reduction (leaves details out) · Pragmatics (made for a purpose, audience, time) · Descriptive (what is) vs Prescriptive (what should be)
Design	new <code>ConcreteClass()</code> inside a high-level class violates Dependency Inversion → introduce an interface and inject it. New formats without editing old code → interface + one implementation per format (Open-Closed)
Domain-driven design	Core Domain = the organisation's main competitive advantage (never outsource it)
Maintenance	adapts software to changed requirements and technology (not CPU temperature)
Quiz tactics	"choose all correct": tick an option if > 50 % sure; grade each option for truth, not style; distractors = impossible for the type · contradict the spec · wrong test level

7. Fill-in-the-blank tokens (exact spelling counts)

`@Test` · `@BeforeEach` `@AfterEach` · `@BeforeAll` `@AfterAll` (static) · `@Disabled` · `assertEquals(expected, actual)` · Setup → Execute → Verify → Teardown · Arrange · Act · Assert ·
`@ExtendWith(EasyMockExtension.class)` · `@TestSubject` · `@Mock` · `expect(...).andReturn(...)` ·
`expectLastCall()` · `andThrow(...)` · `replay(mock)` · `verify(mock)` · record → replay → exercise → verify ·
Statement / Branch / Term / Path coverage · Cyclomatic complexity · Verification · Validation · Dummy / Stub / Spy / Mock / Fake · Unit / Integration / System test · Working Directory → Staging Area (Index) → Repository · `git checkout -b` · `git fetch` · `git push -u origin` · `gradlew build` · `set -e` · Generalisation · Realisation · Dependency · Aggregation · Composition · «include» · «extend» · Fork/Join · Merge · Open-Closed Principle · Strategy · Adapter · Observer · Template Method.

8. The nine lines (last page of your guide)

1. Commit and push before you write anything, then every ten minutes.
2. Unit testing: enum → fields → constructor with guards → getters → calculation → boolean action → tests.
3. One test method can hold `assertTrue(result)` ("did it succeed") **and** `assertEquals(expected, state, delta)` ("what changed").
4. Every double needs a delta. Every `assertEquals` has expected first.
5. The throwing call goes **inside** the `assertThrows` lambda.
6. REST: does `@RequestMapping` already carry a path? Identity in the path, filters in query parameters; POST rejects a supplied id.
7. Deployment: 3-D boxes for nodes, nesting = deployed-on, plain labelled lines between nodes, lollipop/socket only between components.
8. Black-box: classes split per bracket; boundaries are limit−1 / limit / limit+1 around the limit the heading names.
9. Copy required names character-for-character before writing any body.